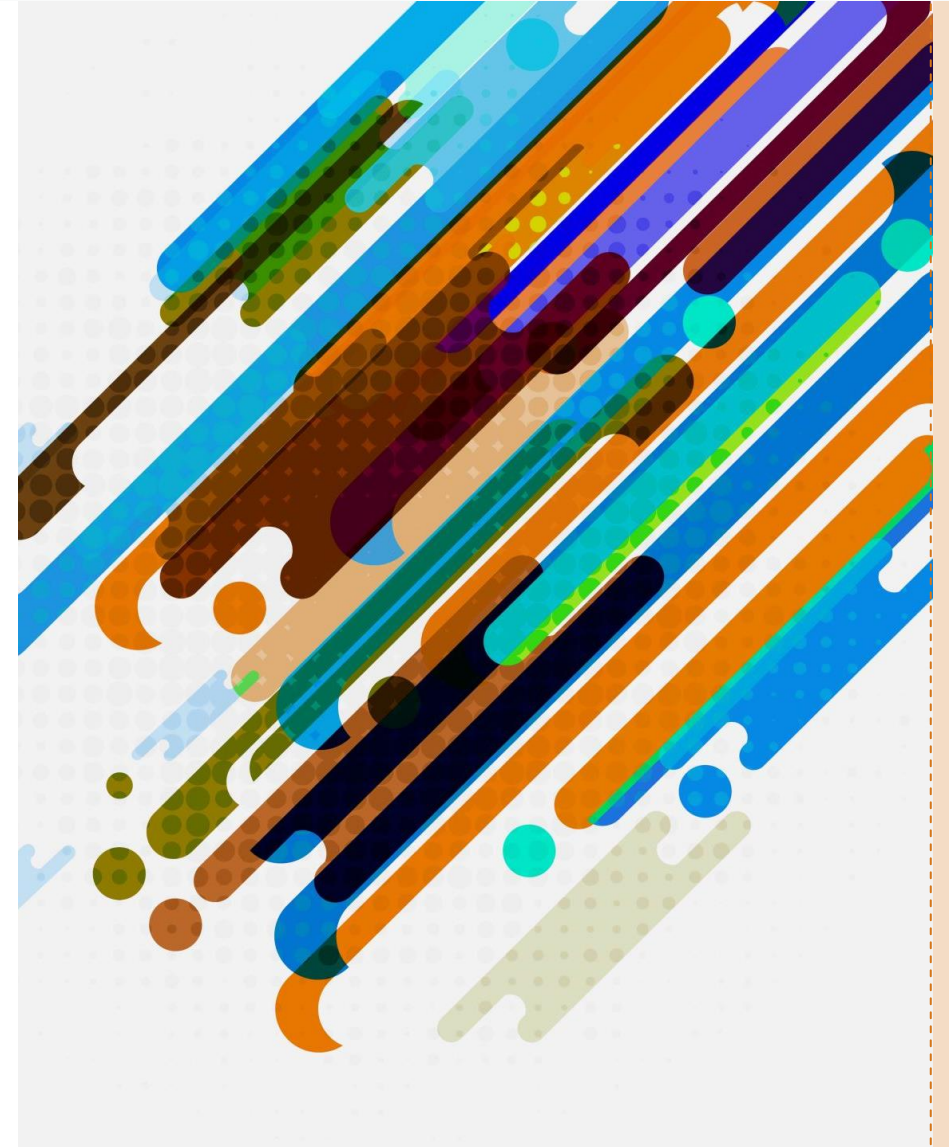


Python Functions

- Topics:
 - Understanding Functions in Python
 - Defining of Functions
 - Parameters and Arguments
 - *args and **kwargs
 - Lambda Functions



Functions

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Functions

Function Syntax :

```
def <function name>(args):  
    .....  
    .....  
    return <value/variable>
```

Creating a Function

In Python a function is defined using the **def** keyword:

Examples

```
def <function name>(args):  
    .....  
    .....  
    return <value/variable>
```

Examples

```
def my_function():  
    print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

```
my_function()
```

Result:

"Hello from a function"

Functions

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

```
def my_function(name):  
    print("welcome to " + name)  
my_function("vamsi")
```

Result:

"Welcome to vamsi"

Functions

Parameters or Arguments?

The terms *parameter* and *argument* can be used for the same thing: information that are passed into a function.

From a function's perspective:

A parameter is the variable listed inside the parentheses in the function definition.

An argument is the value that is sent to the function when it is called.

Functions

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

Example

Example

If the number of arguments is unknown, add a * before the parameter name:

#defining the function

```
def testfunc(*args):  
    print(type(args))
```

#calling the function

```
testfunc(10,20,30,40,50,60)
```

Output :

```
<class 'tuple'>
```

Example

```
def addnum(*args):  
    return sum(args)
```

```
addnum(10,20,30,40,50)
```

Result:

150

Functions

Arbitrary Keyword Arguments, `**kwargs`

If you do not know how many keyword arguments that will be passed into your function, add two asterisk: `**` before the parameter name in the function definition.

This way the function will receive a *dictionary* of arguments, and can access the items accordingly:

```
def my_function(**kwargs):  
    print(type(kwargs))
```

```
my_function(x=10,y=20)
```

```
Result : <class 'dict'>
```

Example

```
def addnum(**kwargs):  
    return sum(kwargs.values())
```

```
addnum(x=10,y=200,a=100,b=50)
```

Result: 360

Python Lambda

A lambda function is a small anonymous function.

A lambda function can take any number of arguments but can only have one expression.

Syntax:

`lambda arguments : expression`

The expression is executed, and the result is returned:

Example

```
x = [100,256,144,64,289]  
list(map(lambda i:i**0.5,x))
```

Result:

```
[10.0,16.0,12.0,8.0,17.0]
```